



jak bude vypadat strom všech C souborů? Zamysli se a navrhni strom co nejchytřeji a nejjednodušeji optimalizovaný.

Pro takto komplexní projekt (kombinující I2S zvuk, Wi-Fi, NTP, ESP-NOW a Deep Sleep) je naprosto klíčové opustit styl "všechno v jednom dlouhém souboru" a přejít na **modulární architekturu**.

Pokud kód rozdělíte logicky podle funkcí, bude se vám snadno ladit (např. můžete testovat zvuk, aniž byste řešili spánek) a nedojde ke kolizi knihoven pro Wi-Fi a ESP-NOW.

Zde je návrh optimálního, chytrého a plochého stromu C/C++ souborů (ideální pro vývoj v PlatformIO nebo ESP-IDF):

Strom souborů

```
Intercom_Project/
├── include/
│   └── config.h           # Jediné místo pro všechna nastavení (piny, hesla)
├── src/
│   ├── main.cpp          # Hlavní smyčka a stavový automat (State Machine)
│   ├── audio.cpp         # Inicializace I2S, čtení mikrofону, zápis do repro
│   ├── audio.h
│   ├── network_espnw.cpp # Odesílání a příjem zvukových paketů (ESP-NOW)
│   ├── network_espnw.h
│   ├── power_time.cpp    # Wi-Fi připojení, NTP synchronizace a Deep Sleep
│   ├── power_time.h
│   ├── hardware_io.cpp   # Obsluha Push-to-Talk tlačítka (debounce)
│   ├── hardware_io.h
│   └── platformio.ini    # Konfigurace projektu a knihoven
```

Význam jednotlivých modulů

1. config.h (Centrální mozek nastavení)

Tento hlavičkový soubor bude obsahovat **pouze** definice (#define). Žádnou logiku.

Když budete měnit piny, heslo na Wi-Fi, MAC adresu druhé vysílačky nebo čas aktivního okna (např. z 11:00 na 12:00), uděláte to pouze zde. Zabráníte tím lovení "magických čísel" po celém kódu.

2. `power_time.cpp / .h` (Správce energie a času)

Tento modul má na starosti to nejdůležitější pro výdrž baterie:

- Funkci pro krátké zapnutí Wi-Fi a stažení NTP času.
- Funkci pro výpočet času, který zbývá do konce aktivního okna.
- Příkazy pro uložení proměnných do RTC paměti a uspání čipu (`esp_deep_sleep_start()`).

3. `network_espnow.cpp / .h` (Komunikační vrstva)

Zde bude veškerá logika pro komunikaci vysílaček.

- Konfigurace ESP-NOW protokolu a přidání MAC adresy druhého zařízení (tzv. párování).
- Funkce (callback) pro příjem zvukového paketu (max 250 bytů) z druhé vysílačky.
- Funkce pro rychlé odeslání zachyceného zvuku.

4. `audio.cpp / .h` (Zpracování I2S)

Modul, který mluví přímo s INMP441 a MAX98357A.

- Nastavení komunikačních linek (I2S sběrnice).
- Funkce `read_microphone()`, která naplní buffer daty (zvukem).
- Funkce `play_speaker()`, která přijatá data pošle do zesilovače.

5. `hardware_io.cpp / .h` (Fyzické ovládání)

Stará se o tlačítko. Převádí fyzický stisk na logický povel. Musí obsahovat tzv. "debouncing" (ošetření zákmitů), aby systém nezaznamenal stisk tlačítka třikrát, když ho zmáčknete jen jednou. Může také ovládat stavovou LED diodu.

6. `main.cpp` (Dirigent)

Hlavní soubor nedělá žádnou složitou práci. Pouze volá funkce z ostatních modulů na základě stavového automatu.

Příklad logiky v `main.cpp`:

1. Je studený start? Zavolej `power_time::syncNTP()`.
2. Je čas 11:00 - 14:00? Zavolej `network_espnow::init()`.
3. Je zmáčknuťo tlačítko? Zavolej `audio::read_microphone()` a hned potom `network_espnow::send_audio()`.
4. Je čas 14:00? Zavolej `power_time::goToDeepSleep()`.

Proč je toto uspořádání nejchytřejší

1. **Zamezení konfliktů:** Wi-Fi knihovny se hádají s ESP-NOW knihovnami. Tím, že Wi-Fi izolujete v `power_time.cpp` a ESP-NOW v `network_espnw.cpp`, předejdete pádům procesoru.
2. **Škálovatelnost:** Pokud se v budoucnu rozhodnete přidat modul SD karty pro nahrávání hovorů, jednoduše přidáte `storage.cpp` a nepřekopete tím zbytek kódu.
3. **Znovupoužitelnost:** Kód pro `audio.cpp` můžete vzít a použít v jakémkoliv dalším projektu s ESP32-S3 a INMP441, aniž byste sebou táhli balast pro interkom a ESP-NOW.

V jakém programovacím prostředí (IDE) plánujete tento kód psát?